

《人工智能导论》Python 语法与 AI 库速查

0. 考试高频判断点

- Python 变量是“名字绑定对象”，不是 C++ 那种变量盒子。
- **可变对象**：list / dict / set / ndarray / DataFrame，函数内可原地修改。
- **不可变对象**：int / float / str / tuple / bool / None。
- = 是重新绑定变量，不一定修改原对象。
- is 判断是否同一个对象；== 判断值是否相等。
- 切片左闭右开：a[start:end] 不包含 end。
- **Scikit-learn** 统一接口：fit / predict / transform / score。
- **Pandas** 常用查看：head / tail / info / describe / shape。

1. Python 基础语法

```
# ----- 基本语法 -----
x = 10                # 不需要声明类型
name = "Alice"
flag = True           # True/False 首字母大写
nothing = None        # 类似 null/nil
# 常见假值
False, None, 0, 0.0, "", [], {}, set()
# 输入输出
print("x =", x)        # x = 10
print(f"name={name}, x={x}") # name=Alice, x=10
# 运算符
print(5 / 2)           # 2.5, 真除法
print(5 // 2)          # 2, 整除
print(5 ** 2)          # 25, 乘方
print(5 != 3)          # True
# and/or 返回的不一定是 bool, 而是某个操作数
print(0 or "hi")       # hi
print(1 and "ok")      # ok
# 循环结构
for i in range(3):
    print(i)            # 0 1 2
for i in range(1, 5, 2):
    print(i)            # 1 3
i = 0
while i < 3:
    i += 1
# range()
range(n)               # [0, n)
range(a, b)            # [a, b)
range(a, b, c)         # 步长 c
```

```

# 切片：左闭右开
a = [0, 1, 2, 3, 4]
print(a[1:4])      # [1, 2, 3]
print(a[:3])       # [0, 1, 2]
print(a[2:])       # [2, 3, 4]
print(a[-1])       # 4
print(a[::-1])     # [4, 3, 2, 1, 0]

# ----- 常用遍历 -----
arr = ["a", "b", "c"]
for i, v in enumerate(arr):
    print(i, v)      # 0 a / 1 b / 2 c
for a, b in zip([1, 2], ["x", "y"]):
    print(a, b)      # 1 x / 2 y

# ----- 函数 -----
def f(): # 多返回值本质是返回 tuple
    return 1, 2
a, b = f()
print(a, b) # 1 2
# lambda
square = lambda x: x * x
print(square(4))    # 16

```

2. Python 容器

```

# list: 可变，类似动态数组
lst = [1, 2, 3]
lst.append(4) # [1, 2, 3, 4]
lst[0] = 99 # [99, 2, 3, 4]

# tuple: 不可变
t = (1, 2, 3)
# t[0] = 9          # 错误: tuple 不可修改

# dict: 键值对
d = {"name": "Alice", "age": 20}
print(d["name"])      # Alice
print(d.get("score", 0)) # 0, 键不存在时返回默认值
d["age"] = 21
d["score"] = 95
for k, v in d.items(): # 遍历
    print(k, v)

# set: 集合，自动去重
s = {1, 2, 2, 3}

```

```

print(s)          # {1, 2, 3}
print(2 in s)     # True

# [!Feature] 赋值不是拷贝
a = [1, 2, 3]
b = a
b.append(4)
print(a)          # [1, 2, 3, 4]

# 浅拷贝
c = a.copy()
c.append(5)
print(a)          # [1, 2, 3, 4]
print(c)          # [1, 2, 3, 4, 5]

# 函数传参: 对象引用/绑定
def modify_list(data):
    data.append(4)    # 修改原 list
    data = [5, 6]    # 只是让局部变量 data 指向新 list
my_list = [1, 2, 3]
modify_list(my_list)
print(my_list)     # [1, 2, 3, 4]

# 可变默认参数陷阱
def bad_append(x, arr=[]):
    arr.append(x)
    return arr
print(bad_append(1))    # [1]
print(bad_append(2))    # [1, 2], 不是 [2]
def good_append(x, arr=None):
    if arr is None:
        arr = []
    arr.append(x)
    return arr

# 列表/字典/集合推导式
nums = [1, 2, 3, 4]
squares = [x * x for x in nums]
print(squares)          # [1, 4, 9, 16]
evens = [x for x in nums if x % 2 == 0]
print(evens)            # [2, 4]
d = {x: x * x for x in nums}
print(d)                # {1:1, 2:4, 3:9, 4:16}
unique = {x % 2 for x in nums}
print(unique)           # {0, 1}

# == vs is
a = [1, 2]
b = [1, 2]

```

```
print(a == b)          # True, 值相等
print(a is b)          # False, 不是同一个对象
```

3. NumPy 速查

```
import numpy as np

# ----- 创建数组 -----
a = np.array([1, 2, 3])
b = np.array([[1, 2, 3], [4, 5, 6]])
print(a.shape)        # (3,)
print(b.shape)        # (2, 3)
print(b.ndim)         # 2
print(b.dtype)        # int64 或 int32
zeros = np.zeros((2, 3))
ones = np.ones((2, 3))
eye = np.eye(3)
r = np.arange(0, 10, 2)      # [0 2 4 6 8]
lin = np.linspace(0, 1, 5)   # [0.  0.25 0.5  0.75 1. ]

# ----- 向量化运算 -----
x = np.array([1, 2, 3])
print(x + 1)           # [2 3 4]
print(x * 2)           # [2 4 6]
print(x ** 2)          # [1 4 9]
# 注意: NumPy 中 * 是逐元素乘法
A = np.array([[1, 2], [3, 4]])
B = np.array([[10, 20], [30, 40]])
print(A * B)           # 逐元素乘法
print(A @ B)           # 矩阵乘法
print(np.dot(A, B))    # 矩阵乘法

# ----- 索引与切片 -----
arr = np.array([[1, 2, 3], [4, 5, 6]])
print(arr[0, 1])       # 2
print(arr[0])          # [1 2 3]
print(arr[:, 1])       # [2 5]
print(arr[0:2, 1:3])   # [[2 3], [5 6]]
# 布尔索引
x = np.array([1, 2, 3, 4])
print(x[x > 2])        # [3 4]
# 聚合函数
m = np.array([[1, 2, 3], [4, 5, 6]])

print(m.sum())         # 21
print(m.mean())        # 3.5
```

```

print(m.max())          # 6

print(m.sum(axis=0))     # [5 7 9], 按列聚合
print(m.sum(axis=1))     # [6 15], 按行聚合

# ----- 形状变换 -----
x = np.arange(6)
print(x.reshape(2, 3))  # [[0 1 2], [3 4 5]]

# ----- 拼接 -----
a = np.array([[1, 2]])
b = np.array([[3, 4]])

print(np.concatenate([a, b], axis=0)) # 按行拼接: [[1 2], [3 4]]
print(np.concatenate([a, b], axis=1)) # 按列拼接: [[1 2 3 4]]

# ----- 随机数 -----
np.random.seed(0)
print(np.random.rand(2, 3))          # 0~1 均匀分布
print(np.random.randn(2, 3))         # 标准正态分布
print(np.random.randint(0, 10, size=5))

```

4. Pandas 速查

```

import pandas as pd

# ----- 创建 DataFrame -----
df = pd.DataFrame({
    "name": ["Alice", "Bob", "Cindy"],
    "age": [20, 21, 19],
    "score": [90, 85, 95]
})

# ----- 查看数据 -----
print(df.head())          # 前 5 行
print(df.head(3))         # 前 3 行
print(df.tail(2))         # 后 2 行
print(df.shape)           # (行数, 列数)
print(df.columns)         # 列名
print(df.info())          # 数据类型、缺失值
print(df.describe())      # 数值列统计信息

# ----- 选择列 -----
print(df["age"])           # 单列, Series
print(df[["name", "score"]]) # 多列, DataFrame

```

```

# ----- 选择行 -----
print(df.iloc[0])          # 按位置取第 0 行
print(df.iloc[0:2])        # 按位置取前 2 行
print(df.loc[0])           # 按标签取第 0 行

# iloc: 位置索引; loc: 标签索引
print(df.iloc[0, 1])       # 第 0 行第 1 列
print(df.loc[0, "age"])    # 标签为 0 的行, age 列

# ----- 条件筛选 -----
print(df[df["age"] > 20])

# 多条件必须用 & / |, 每个条件加括号
print(df[(df["age"] > 19) & (df["score"] >= 90)])

# ----- 新增/修改列 -----
df["passed"] = df["score"] >= 60
df["score2"] = df["score"] + 5

# ----- 排序 -----
print(df.sort_values("score"))          # 升序
print(df.sort_values("score", ascending=False)) # 降序

# ----- 缺失值 -----
df2 = pd.DataFrame({
    "a": [1, None, 3],
    "b": [4, 5, None]
})

print(df2.isna())          # 判断缺失
print(df2.dropna())        # 删除含缺失值的行
print(df2.fillna(0))       # 缺失值填 0

# ----- 统计 -----
print(df["score"].mean())
print(df["score"].max())
print(df["score"].min())
print(df["score"].value_counts())

# ----- 分组 -----
df3 = pd.DataFrame({
    "class": ["A", "A", "B", "B"],
    "score": [90, 80, 70, 60]
})

print(df3.groupby("class")["score"].mean())

# ----- 读写文件 -----
# df = pd.read_csv("data.csv")
# df.to_csv("out.csv", index=False)

```

```
# ----- 合并 -----
left = pd.DataFrame({"id": [1, 2], "name": ["A", "B"]})
right = pd.DataFrame({"id": [1, 2], "score": [90, 80]})

merged = pd.merge(left, right, on="id")
print(merged)
```

5. Matplotlib 速查

```
import matplotlib.pyplot as plt

# ----- 折线图 -----
x = [1, 2, 3, 4]
y = [2, 4, 6, 8]

plt.figure()
plt.plot(x, y, label="line")
plt.xlabel("x")
plt.ylabel("y")
plt.title("Line Plot")
plt.legend()
plt.grid(True)
plt.show() # 显示图像，不是清空图像

# ----- 散点图 -----
plt.figure()
plt.scatter(x, y)
plt.title("Scatter Plot")
plt.show()

# ----- 柱状图 -----
names = ["A", "B", "C"]
scores = [90, 80, 95]

plt.figure()
plt.bar(names, scores)
plt.title("Bar Chart")
plt.show()

# ----- 直方图 -----
data = [1, 1, 2, 2, 2, 3, 4]

plt.figure()
plt.hist(data, bins=4)
plt.title("Histogram")
```

```
plt.show()

# ----- 清空/关闭图像 -----
plt.clf()      # 清空当前 figure
plt.cla()      # 清空当前 axes
plt.close()    # 关闭当前 figure

# ----- 面向对象写法 -----
fig, ax = plt.subplots()
ax.plot(x, y)
ax.set_title("00 Style")
ax.set_xlabel("x")
ax.set_ylabel("y")
plt.show()

# ----- 常见考试点 -----
# plt.plot()      折线图
# plt.scatter()   散点图
# plt.bar()       柱状图
# plt.hist()      直方图
# plt.show()      显示图
# plt.clf()       清空当前图
# plt.close()     关闭图
```

6. Scikit-learn 速查

```
# ----- 基本接口 -----
# model.fit(X, y)      训练模型
# model.predict(X)     预测
# model.score(X, y)    评估
# transformer.fit(X)   学习预处理参数
# transformer.transform(X) 转换数据
# transformer.fit_transform(X) 先 fit 再 transform

# ----- 标准监督学习流程 -----
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.metrics import accuracy_score, mean_squared_error, r2_score

# X 必须通常是二维: [样本数, 特征数]
# y 通常是一维: [样本数]

X = [
    [1, 2],
    [2, 3],
```



```

    [3, 4],
    [4, 5]
]
y = [3, 5, 7, 9]

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.25,
    random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)      # 训练
y_pred = model.predict(X_test)  # 预测

print(model.score(X_test, y_test))  # 一般回归默认 R^2

# ----- 标准化 -----
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)  # 训练集 fit + transform
X_test_scaled = scaler.transform(X_test)        # 测试集只 transform

# ----- 分类示例 -----
X = [[0], [1], [2], [3]]
y = [0, 0, 1, 1]

clf = LogisticRegression()
clf.fit(X, y)

pred = clf.predict([[1.5], [3]])
print(pred)                # 预测类别, 如 [0 1]
print(clf.score(X, y))     # 分类默认 accuracy

# ----- 评价指标 -----
# 分类:
# accuracy_score(y_true, y_pred)
# precision / recall / f1_score / confusion_matrix

# 回归:
# mean_squared_error(y_true, y_pred)
# r2_score(y_true, y_pred)

# ----- Pipeline -----
from sklearn.pipeline import Pipeline
from sklearn.svm import SVC

pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("model", SVC())
])

```

```
pipe.fit(X_train, y_train)
pipe.predict(X_test)

# ----- 常见模型 -----
# LinearRegression      线性回归
# LogisticRegression    逻辑回归, 常用于分类
# KNeighborsClassifier  KNN 分类
# DecisionTreeClassifier 决策树分类
# RandomForestClassifier 随机森林分类
# SVC                   支持向量机分类
# KMeans                 聚类, 无监督学习

# ----- 高频考试点 -----
# fit(X, y)              训练监督学习模型
# predict(X)             预测结果
# transform(X)           数据预处理转换
# fit_transform(X)       预处理时常见
# score(X, y)            模型评估
```

7. 常见选择题/判断题速记

```
# 1. list 是可变对象, 函数内 append 会影响外部
def f(a):
    a.append(100)

x = [1, 2]
f(x)
print(x)          # [1, 2, 100]

# 2. 重新赋值不会影响外部变量绑定
def g(a):
    a = [9, 9]

x = [1, 2]
g(x)
print(x)          # [1, 2]

# 3. Pandas 前 n 行
# df.head(n)

# 4. Pandas 后 n 行
# df.tail(n)

# 5. Matplotlib 显示图
# plt.show()
```

```
# 6. Matplotlib 清空图
# plt.clf() / plt.cla()

# 7. Scikit-learn 训练模型
# model.fit(X, y)

# 8. Scikit-learn 预测
# model.predict(X)

# 9. Scikit-learn 预处理转换
# transformer.transform(X)

# 10. Scikit-learn 训练集划分
# train_test_split(X, y, test_size=..., random_state=...)

# 11. NumPy 数组形状
# arr.shape

# 12. NumPy 按列聚合
# arr.sum(axis=0)

# 13. NumPy 按行聚合
# arr.sum(axis=1)

# 14. Python 判断值相等
# a == b

# 15. Python 判断是否同一对象
# a is b
```

8. 和 C++ / Go 经验不同的重点

```
# ----- 变量是名字绑定，不是固定类型盒子 -----
x = 1
x = "hello"      # 合法

# ----- 没有 ++ / -- -----
x = 1
x += 1           # 正确
# x++           # 错误

# ----- for 主要遍历可迭代对象 -----
for v in [10, 20, 30]:
    print(v)
```

```
# ----- range(3) 是 0,1,2, 不包含 3 -----
for i in range(3):
    print(i)

# ----- 字符串不可变 -----
s = "abc"
# s[0] = "A"      # 错误
s = "A" + s[1:]   # 正确

# ----- list 切片会产生新 list -----
a = [1, 2, 3]
b = a[:]
b.append(4)
print(a)          # [1, 2, 3]

# ----- 函数可以返回多个值，本质是 tuple -----
def foo:
    pass

def foo():
    return 1, 2

a, b = foo()

# ----- Python 没有 switch/case 的传统写法 -----
# 新版本有 match-case，但考试基础题通常更常用 if-elif-else

# ----- 异常处理 -----
try:
    x = int("123")
except ValueError:
    print("convert error")
finally:
    print("done")

# ----- import -----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.linear_model import LinearRegression
```